

せいきひょうげん 正規表現

プログラミングを勉強している^{べんきょう}と必ず^{かなら}出会う^{で あ}、魔法^{まほう}の言葉^{ことば}のような記号^{きごう}の羅列^{られつ}。それが「正規表現^{せいきひょうげん}」です。
英語^{えいご}で "Regular Expression" ^いと言います。

JavaScriptでは、**RegExp**（レグエクスプ）オブジェクト^{つか}を使います。

RegExpは「魔法^{まほう}の虫めがね^{むし}」です。たくさんの文字^{も じ}の中から、自分の探^なきたい「パターン^{じぶん さが}」だけをピカッと^{ひか}光^みらせて見^みつけることができます。

I. 「正規表現^{せいきひょうげん}」ってなに？

普通の検索^{ふつう けんさく}（**indexOf** など）は、「特定^{とくてい}の言葉^{ことば}」を探^{さが}します。

RegExpは、「言葉^{ことば}のパターン^{さが}（ルール）」を探^{さが}します。

- 普通の検索^{ふつう けんさく}：「『田中^{たなか}』さんを探^{さが}して」
- **RegExp**：「『数字^{すうじ}が3つ、ハイフン、数字^{すうじ}が4つ』の形^{かたち}（電話番号^{でんわばんごう}）を探^{さが}して」

このように、あやふやなルールで検索^{けんさく}できるのがすごいところです。

II. RegExpオブジェクトの作り方^{つく かた}

2つの書き方^{か かた}がありますが、よく使われるのは **/ /**（スラッシュ）で囲む方法^{つか ほうほう}です。

```
// 1. スラッシュで囲む（リテラル） ★おすすめ
const pattern1 = /abc/;

// 2. new を使う
const pattern2 = new RegExp("abc");
```

III. よく使う「魔法^{まほう}の記号^{きごう}」（1）

正規表現^{せいきひょうげん}（RegExp）のメタ文字^{も じ}は、文字^{も じ}の「種類^{しゅるい}」や「数^{かず}」を指定^{して い}する特別なルール^{とくべつ}です。

「トランプのジョーカー^{さんすう}」や「算数^{たど}のルール^{せつめい}」に例えて説明^{たの}すると、パズル^{りかい}のように楽しく理解^{りかい}できます。

よく使われる7つの記号^{きごう}（メタ文字^{も じ}）を、やさしく解説^{かいせつ}します！

1. . （ドット）

→ 「なんでもいい一文字」

- イメージ：トランプの「ジョーカー」や、クイズの「穴埋（あなう）め」です。
- ルール：数字でも、漢字でも、アルファベットでも、**「とにかく1文字」**あればOK。

れい

例：/あ.い/

- * 「あい」 ... OK
- * 「ああい」 ... OK
- * 「あ1い」 ... OK
- * 「あい」 ... NG（文字が足りない）

2. *（アスタリスク）

→ 「0回以上のくり返し」

- イメージ：おかわり自由。「なくてもいいし、無限（むげん）にあってもいい」。
- ルール：直前（ちよくぜん）の文字が「0個、1個、2個.....」と、いくつ続いていてもOK。

れい

例：/あ*い/

- * 「い」 ... OK（「あ」が0個でもいい）
- * 「あい」 ... OK
- * 「あああい」 ... OK

3. +（プラス）

→ 「1回以上のくり返し」

- イメージ：最低（さいてい）でも1つは必要。「1個以上なら、いくつでもいい」。
- ルール：直前（ちよくぜん）の文字が「1個、2個、3個.....」と続いていればOK。

れい

例：/あ+い/

- * 「い」 ... NG（「あ」が最低1つは必要！）
- * 「あい」 ... OK
- * 「あああい」 ... OK

4. ?（クエスチョン）

→ 「0回か1回だけ」

- イメージ：「あってもなくてもいい（最大1つまで）」。
- ルール：直前（ちよくぜん）の文字が「ない」か「1個だけある」状態。

れい

例：/あ?い/

- * 「い」 ... OK

- * 「あい」 ... OK
- * 「ああい」 ... **NG** (2個以上はダメ)

5. ^ (キャレット)

→ 「行がここから始まる」

- イメージ：マラソンの「スタートライン」です。
- ルール：その文字で文章が始まっていることをチェックします。

例：/^おはよう/

- * 「おはようございます」 ... OK
- * 「みんな、おはよう」 ... **NG** (「みんな」から始まっているから)

6. \$ (ドル)

→ 「行の最後がこれでおわる」

- イメージ：マラソンの「ゴールテープ」です。
- ルール：その文字で文章が終わっていることをチェックします。

例：/です\$/

- * 「田中です」 ... OK
- * 「田中ですよ」 ... **NG** (「よ」で終わっているから)

7. | (パイプ・縦線)

→ 「または (OR)」

- イメージ：レストランのメニュー。「A または B どちらか選んで」というルール。
- ルール：左側か右側、どちらかの文字があればOK。

例：/りんご|みかん/

- * 「りんご」 ... OK
- * 「みかん」 ... OK
- * 「バナナ」 ... NG

覚え方のまとめテーブル

記号	意味	覚え方のヒント
.	なんでも1文字	何が入るかワクワクする「穴」
*	0個以上	おかわり自由 (なくてもいい)
+	1個以上	必ず1つは持ってきて！

きごう 記号	いみ 意味	おぼ かた 覚え方のヒント
?	0か1個	オプション（あってもなくてもいい）
^	始まり	文の最初にあるスタートライン
\$	終わり	文の最後にあるゴールテープ
	または	AかB、どちらか好きな方！

いかがでしょうか？記号の「パワー」を知ると、短いコードで複雑なチェックができるようになります。

4. ? (クエスチョン)

→ 「0回か 1回だけ」

- イメージ：「あってもなくてもいい（最大1つまで）」。
- ルール：直前の文字が「ない」か「1個だけある」状態。

例：/あい/* 「い」 ... OK * 「あい」 ... OK * 「ああい」 ... NG (2個以上はダメ)

5. ^ (キャレット)

→ 「行がここから始まる」

- イメージ：マラソンの「スタートライン」です。
- ルール：その文字で文章が始まっていることをチェックします。

例：/^おはよう/* 「おはようございます」 ... OK * 「みんな、おはよう」 ... NG (「みんな」から始まっているから)

6. \$ (ドル)

→ 「行の最後がこれでおわる」

- イメージ：マラソンの「ゴールテープ」です。
- ルール：その文字で文章が終わっていることをチェックします。

例：/です\$/ * 「田中です」 ... OK * 「田中ですよ」 ... NG (「よ」で終わっているから)

7. | (パイプ・縦線)

→ 「または (OR)」

- イメージ：レストランのメニュー。「A または B どちらか選んで」というルール。
- ルール：左側か右側、どちらかの文字があればOK。

例： `/りんご|みかん/` * 「りんご」 ... OK * 「みかん」 ... OK * 「バナナ」 ... NG

覚え方のまとめテーブル

記号	意味	覚え方のヒント
.	なんでも1文字	何が入るかワクワクする「穴」
*	0個以上	おかわり自由（なくてもいい）
+	1個以上	必ず1つは持ってきて！
?	0か1個	オプション（あってもなくてもいい）
^	始まり	文の最初にあるスタートライン
\$	終わり	文の最後にあるゴールテープ
	または	AかB、どちらか好きな方！

いかがでしょうか？ 記号の「パワー」を知ると、短いコードで複雑なチェックができるようになります。

IV. よく使う「魔法の記号」（2）

正規表現（RegExp）のメタ文字、さらに便利なものを紹介します！ 今回は、文字を「グループ化」したり、「回数を正確に指定」したりする、少し高度なルールです。

「チーム作り」や「回数の予約」に例えて説明します。

1. `()`（カッコ）

→ 「チーム（グループ）を作る」

- イメージ：複数の文字をひとつにまとめて「1つのチーム」として扱います。
- ルール：チームに対して、さっき勉強した `*` や `+` を使うことができます。

例： `/(abc)+/` * 「abc」 ... OK * 「abcabcabc」 ... OK（「abc」というチームが繰り返されている） * 「abcc」 ... NG（チーム全員で繰り返さないとダメ）

2. `[]`（角カッコ）

→ 「この中のどれか1文字」

- イメージ：「この中（リスト）にある文字なら、誰でもいいよ」というメンバー名簿です。
- ルール：カッコの中に入っている文字のどれか1つに合えばOK。

例： `/[ABC]<ん/`

- 「Aくん」 ... OK
- 「Bくん」 ... OK
- 「Cくん」 ... OK
- 「Dくん」 ... NG (名簿に名前がない)

このように[]で囲まれて『この中のどれか1つに合えばOK』という文字のグループを**文字クラス**といいます。

文字クラスにはいくつかのバリエーションがあります。

- **文字クラスの先頭に^をつけると、文字クラスの否定を表します。**

例: `/[^ABC]くん/`

- 「Aくん」 ... NG
- 「Bくん」 ... NG
- 「Cくん」 ... NG
- 「Dくん」 ... OK (名簿に名前がないものがOK)

- **数字やアルファベットが連続しているとき、最初の文字と最後の文字を-でつなげて、短く書くことができます。**

例: `/[A-C]くん/`

- 「Aくん」 ... OK
- 「Bくん」 ... OK
- 「Cくん」 ... OK
- 「Dくん」 ... NG (AからCのあいだにない文字)

3. {n} {n,} {n, m} (波かっこ)

→ 「回数を数字で指定する」

これらは+や*よりも、もっと細かく「何回か」を予約する記号です。

{n} : ぴったり n回

- 例: `/\d{3}/` (数字がぴったり3つ)
- 「123」はOK、「12」はNG。

{n,} : n回以上 (いくらでも)

- 例: `/\d{3,}/` (数字が3つ以上)
- 「123」も「12345」もOK。

{n, m} : n回から m回のあいだ

- 例： `/\d{3,5}/` (数字が3つ、4つ、または5つ)
- 「1234」はOK、「123456」はNG。

4. \ (バックスラッシュ / 円マーク)

→ 「魔法を消して、普通の文字に戻す」

- イメージ：「エスケープ (逃がす)」といいます。特別な力を持っているメタ文字から、魔法を取り上げます。
- ルール：メタ文字の前に書くと、ただの「記号」として探せるようになります。

例：普通のドット (.) を探したいとき * . だけだと「なんでも1文字」という魔法が発動します。 * `/\./` と書くと、文章の中にある ** 「点 (ドット)」 ** そのものを探せます。

まとめテーブル (留学生向け)

記号	意味	覚え方のヒント
()	グループ化	「みんなで1つのチームだよ！」
[]	どれか1つ	「このリストの中の誰か1人！」
{n}	ぴったり n回	「回数を予約 (よやく) する」
{n, m}	n回 ~ m回	「最小と最大を決める」
\	エスケープ	「魔法をキャンセル！」

いかがでしょうか？これで正規表現の主要なパーツが揃いました。

留学生へのアドバイス

RegExpは、最初は難しくて「暗号」に見えます。でも、全部覚える必要はありません！

「RegExpは『文字の形をチェックするルールブック』だよ。
郵便番号、メールアドレス、パスワードの強さ (英数字が入っているか) を調べるときに、世界中のプログラマーが使っているんだ。
難しいパターンは、インターネットで『RegExp 郵便番号』のように検索してコピーすれば大丈夫だよ！」

[!IMPORTANT]

スラッシュ / / の後ろにつけるオプション (フラグ)

- * `/abc/i` : 大文字と小文字を区別しない (ignore)。
- * `/abc/g` : 全部探す (global)。

この2つを知ると、さらに便利に使ってもらえます。

いかがでしょうか？「完璧なパターンを自分で書く」ことよりも、「ルールで検索できる便利な道具がある」ことを知ることが第一歩です！

V. 実際に使ってみよう

1. Stringオブジェクトのmatch()メソッドと連携

上の例では、RegExpオブジェクトのtest()メソッドを使って、そのパターンが含まれているかどうかをチェックしました。

しかし、JavaScriptで文字列の中から特定の文字を探し出すときに、一番よく使われるのは、「RegExpオブジェクト（パターン）」と「Stringメソッドのmatch()」の連携という、強力な組み合わせです。

RegExpのtest()やexec()はパターンが主語でしたが、match()は文字列が主語になります。

- 書き方の形：文字列もじれつ.match(パターン)

例えば：「この文章のなかで（文字列）、このルールに合うものを（パターン）、集めてちょうだい！（match）」

例

```
const myRegExp = new RegExp("app.*"); // 「app.*」というパターンがあるか？
const text = "I have an apple.";

mat = text.match(myRegExp);
if (mat !== null)
  console.log("見つかりました！");
console.log(mat); // ['apple.', index: 10, input: 'I have an apple.', groups: undefined]
```

match()メソッドの戻り値は配列で、上の場合

- マッチした最初の文字列 (apple)
- マッチした文字の位置 (10)
- 調べた対象の文字列 ('I have an apple.')

です（groupsについての解説は省略）。

2. 正規表現のgフラグによる動きの変化

match()の本当の面白さは、正規表現の後ろにつける**g（グローバルフラグ：全部探すという意味）**があるかないかで、返ってくる結果の形（データ型）がガラリと変わることです。

g フラグをとく：合うものを全部集めてリストに

文字列の中を上から下まで全部調べて、パターンに合う言葉を「ただのきれいな配列（リスト）」にして全部返してくれます。詳しい情報は入りません。

```
const text = "私の学校には、Aクラス、Bクラス、Cクラスがあります。";

const myRegExpG = new RegExp("クラス", "g"); // うしろに g をつけました！

const resultG = text.match(myRegExpG);
console.log(resultG); // ["クラス", "クラス", "クラス"] (3つぜんぶつかまえた！)
console.log(resultG.length); // 3 (いくつ見つかったかもすぐわかります)
```

[!note] もし文字列の中にパターンに合う言葉が1つもなかった場合、`match()` はどちらの場合でも `**null` (空っぽ) `**` を返します。

3. i フラグの i は何の略？

この i は、英語の **Ignore Case** (イグノア・ケース：大文字小文字を無視する) という言葉の頭文字です。

通常、JavaScriptの正規表現は大文字と小文字を「完全に別の文字」として区別します。しかし、i フラグをつけると、それらを同じ文字として扱ってくれるようになります。

コードで見る違い (フラグがある時・ない時)

自分のパソコンのブランド (Apple) について書いた文章から文字を探してみましょう。

```
const text = "My computer is an apple MacBook."; // 小文字の apple

// 1. i フラグを【つけない】とき (きびしく さがす)
const regExp1 = new RegExp("Apple"); // 大文字の A
console.log(text.match(regExp1) !== null); // false (大文字と小文字がちがうので、見つからない！)

// 2. i フラグを【つける】とき (やさしく さがす)
const regExp2 = new RegExp("Apple", "i"); // うしろに i をつけました！
console.log(text.match(regExp2) !== null); // true (大文字小文字をむしるので、見つかった！)
```

このように、パターンのスラッシュの後ろに `/パターン/i` と書くだけで、Apple も apple も APPLE も、ぜんぶ一緒に見つけてくれるようになります。

前に習った g フラグと一緒に使える？

はい、^{いっしょ}一緒に使えます！^{じゅんばん}順番は^{ig}igでも^{gi}giでもどちらでも大丈夫です。
 これらを^{くあ}組み合わせると、「^{おおもじこもじ}大文字小文字を^き気にせず（ⁱi）、^{ぶんしょうぜんたい}文章全体から^{あつ}すべて集める（^gg）」という
^{きょうりよく}強力な^{そうさ}検索ができます。

```
const text = "JavaScript is great. I love javascript!";

// 大文字小文字を気にせず (i)、ぜんぶ集める (g)
const myRegExp = new RegExp("javascript", "ig");

const result = text.match(myRegExp);
console.log(result); // ["JavaScript", "javascript"] (両方ともしっかり捕まえた！)
```

② ^{おおもじこもじ}i フラグ（大文字小文字を無視）をつけたとき

```
const pattern2 = new RegExp("banana", "i"); // 大文字小文字を気にしない

const result2 = text.replace(pattern2, "りんご");
console.log(result2);
// 結果: "私は りんご が好きです。でも、banana はもっと好きです。"
// ※ 最初に見つかった大文字の "Banana" が置 (お) き換 (か) わりましたが、2つ目の小文字はそのままです。
```

③ ^{りょうほう}g と ^{ぜんぶ}i フラグを【両方】つけたとき：全部まとめて置き換える！

^{ぶんしょう}文章の中にある^{なか}ターゲットを、^{おおもじこもじ}大文字小文字に関わらず^{かか}すべて^{いっばつ}一発で^あ置き換えたいときは、^{gi}gi（または^{ig}ig）を使います。これが実務でも一番よく使われる形です。

```
const pattern3 = new RegExp("banana", "gi"); // 大文字小文字を気にせず、ぜんぶ！

const result3 = text.replace(pattern3, "りんご");
console.log(result3);
// 結果: "私は りんご が好きです。でも、りんご はもっと好きです。"
// ※ みごとにすべてのバナナがりんごに変身（へんしん）しました！
```

3. 実務でよくある使い方：「^{ふてきせつ}NGワード（不適切な言葉）」の^{ひひょうじ}非表示

Webサイトの^{けいじばん}掲示板や^{つか}チャットアプリを作るとき、^{だれ}誰かが^{わるくち}悪口や^{ひみつ}秘密の^{じょうほう}情報を書き込んだら、^こ自動的に^{じどうてき}***
 や^お[<^かruby>^き削除<^{つか}rt>^きさくじょ</rt></ruby>]に置き換えるシステム（フィルター機能）によく使われます。

```
const chatMessage = "お前のバカ！ BAKA！ 本当にもう知らない！";
```

```
// 「バカ」という言葉（ことば）を大文字小文字にかかわらず（gi）すべて見つける
const ngWordPattern = new RegExp("バカ|baka", "gi")

// 安全な文字に置き換える
const safeMessage = chatMessage.replace(ngWordPattern, "***");
console.log(safeMessage);
// 結果: "お前の***！***！本当にもう知らない！"
```

4. まとめ

`String` の `replace()` メソッドは、正規表現と一緒に使うことで、文章の中の文字を自由自在《じゅうじざい》に書き換えることができます。
 大切なルールは次の3つです。

1. 順番：文字列.`replace(/探すパターン/, "新しい文字")` の順番で書きます。
2. フラグのパワー：`/パターン/gi` のように `g` と `i` を一緒につけると、大文字・小文字を気にせず、文章の中にあるターゲットを**全部まとめて**新しい文字に変えてくれます。
3. 元の文字は消えない：`replace()` を使っても、元の変数の中身は変わりません。必ず新しく作った結果を別の変数（`const newText = ...`）に代入《だいにゅう》して使いましょう。

ユーザーが入力したデータから、いらないスペース（空白）を消したり、危険な文字を隠したりするときにとっても役に立つので、ぜひ覚えてくださいね！

② `i` フラグ（大文字小文字を無視）をつけたとき

```
const pattern2 = new RegExp("banana", "i"); // 大文字小文字を気にしない

const result2 = text.replace(pattern2, "りんご");
console.log(result2);
// 結果: "私は りんご が好きです。でも、banana はもっと好きです。"
// ※ 最初に見つかった大文字の "Banana" が置（お）き換（か）わりましたが、2つ目の小文字はそのままです。
```

③ `g` と `i` フラグを【両方】つけたとき：全部まとめて置き換える！

文章の中にあるターゲットを、大文字小文字に関わらずすべて一発で置き換えたいときは、`gi`（または `ig`）を使います。これが実務でも一番よく使われる形です。

```
const pattern3 = new RegExp("banana", "gi"); // 大文字小文字を気にせず、ぜんぶ！

const result3 = text.replace(pattern3, "りんご");
console.log(result3);
```

```
// 結果: "私は りんご が好きです。でも、りんご はもっと好きです。"
// ※ みごとにすべてのバナナがりんごに変身（へんしん）しました！
```

3. 実務でよくある使い方：「NGワード（不適切な言葉）」の非表示

Webサイトの掲示板やチャットアプリを作るとき、誰かが悪口や秘密の情報を書き込んだら、自動的に *** や [<ruby>削除<rt>さくじょ</rt></ruby>] に置き換えるシステム（フィルター機能）によく使われます。

```
const chatMessage = "お前のバカ！BAKA！本当にもう知らない！";

// 「バカ」という言葉（ことば）を大文字小文字にかかわらず（gi）すべて見つける
const ngWordPattern = new RegExp("バカ|baka", "gi")

// 安全な文字に置き換える
const safeMessage = chatMessage.replace(ngWordPattern, "***");
console.log(safeMessage);
// 結果: "お前の***！***！本当にもう知らない！"
```

4. まとめ

String の `replace()` メソッドは、正規表現と一緒に使うことで、文章の中の文字を自由自在《じゅうじざい》に書き換えることができます。
大切なルールは次の3つです。

1. 順番：文字列.`replace(/探すパターン/, "新しい文字")` の順番で書きます。
2. フラグのパワー：`/パターン/gi` のように `g` と `i` を一緒につけると、大文字・小文字を気にせず、文章の中にあるターゲットを**全部まとめて**新しい文字に変えてくれます。
3. 元の文字は消えない：`replace()` を使っても、元の変数の中身は変わりません。必ず新しく作った結果を別の変数（`const newText = ...`）に代入《だいにゅう》して使いましょう。

ユーザーが入力したデータから、いらないスペース（空白）を消したり、危険な文字を隠したりするときにとっても役に立つので、ぜひ覚えてくださいね！